



**МОСКОВСКИЙ
ПОЛИТЕХ**

МИНОБРНАУКИ РОССИИ

Федеральное государственное автономное образовательное учреждение
высшего образования

«Московский политехнический университет»

Факультет информационных технологий

Кафедра «Информационная безопасность
автоматизированных систем»

ОТЧЕТ

по проекту

по дисциплине

«Технология и методы программирования»

Студенты группы 251-372

Орлов Руслан Максимович

Карелин Кирилл Денисович

Серегина Елизавета Денисовна

Воробьева Елизавета Александровна

Сарафанов Алексей Сергеевич

Москва - 2026

В рамках выполнения проекта по дисциплине «Технология и методы программирования» были изучены современные технологии разработки клиент-серверных приложений. Основное внимание уделялось сетевому программированию, работе с базами данных, аутентификации пользователей и документированию кода. В качестве основы использовался проект TIMP-251-372-group-5, размещенный на GitHub. Проект представлял собой клиент-серверное приложение для графического отображения ветвящейся (кусочной) функции с системой авторизации, регистрации, двухфакторной аутентификации и восстановления пароля. В процессе работы были изучены принципы TCP-взаимодействия, потокобезопасной обработки запросов, а также методы тестирования программного обеспечения. Для разработки использовались язык C++17 и фреймворк Qt 5.

Для организации разработки использовались Git и GitHub. Система контроля версий Git позволяла отслеживать изменения в проекте, хранить историю разработки и организовывать совместную работу пяти участников подгруппы. В процессе работы применялись основные команды Git: `git clone`, `git pull`, `git add`, `git commit` и `git push`. Для разработки нового функционала создавались отдельные ветки (`main`, `lan-demo`), после чего изменения объединялись с основной веткой проекта. GitHub использовался как централизованное хранилище проекта, в репозитории размещались исходный код сервера и клиента, документация, UML-диаграммы, тесты и конфигурационные файлы. Также применялся механизм Pull Request, который позволял проверять изменения перед их добавлением в основную версию проекта. Использование GitHub значительно упростило организацию командной разработки и обеспечило безопасное хранение кода.

Для контейнеризации серверной части приложения применялась технология Docker. Использование Docker позволило создать единое окружение разработки и избежать проблем совместимости между различными устройствами. Контейнер использовался для запуска TCP-сервера, который слушал на порту 33333 и обрабатывал запросы клиентов. Внутри контейнера работала база данных SQLite для хранения пользователей, а также SMTP-клиент для отправки кодов подтверждения через Gmail. Конфигурация описывалась в файле Dockerfile на базе Ubuntu 22.04 с установленными библиотеками Qt 5. Сборка выполнялась командой `docker build`, а запуск — командой `docker run` с пробросом порта 33333. На Docker Hub размещался готовый образ `rainkord/timp-server`, что позволяло запустить сервер одной командой. Docker обеспечил переносимость серверного компонента, упростил настройку и ускорил процесс развертывания.

Для разработки проекта использовались язык программирования C++17 и фреймворк Qt 5 (Widgets, Network, SQL, Concurrent). Использование C++ обеспечило высокую производительность серверной части и возможность эффективной работы с числовыми вычислениями. Серверная часть включала TCP-сервер на базе `QTcpServer`, singleton-класс `Database` для работы с SQLite, асинхронную отправку email через `QSslSocket` с использованием `QtConcurrent`, а также класс `Calculator` для вычисления кусочной функции и генерации данных графика. Клиентская часть представляла собой GUI-приложение на Qt Widgets с многоэкранной навигацией через `QStackedWidget`. Реализовывались виджеты авторизации, регистрации (3-шаговый визард), ввода OTP-кода (6-значный с

автопереключением полей), восстановления пароля, построения графика функции с помощью QPainter, а также диалоги просмотра задания и блок-схемы алгоритма. Использование Qt позволило ускорить разработку и обеспечить стабильную работу как серверной, так и клиентской части проекта.

Для документирования исходного кода использовалась система Doxygen. Она позволяла автоматически создавать техническую документацию на основе комментариев в коде. С помощью Doxygen документировались все классы, функции, методы, сигналы, слоты и поля данных — более 480 комментариев на русском языке в формате @brief, @param, @return, @note. Главная страница документации (mainpage.dox) содержала описание архитектуры приложения, таблицы компонентов сервера и клиента, описание протокола обмена данными и перечень используемых технологий. Генерация документации выполнялась командой `doxygen docs/Doxyfile`, результат размещался в `docs/output/html/`. Использование Doxygen позволило поддерживать документацию в актуальном состоянии и повысило читаемость исходного кода.

В проекте использовалась база данных SQLite для хранения информации о пользователях (логин, email, хеш пароля). Класс Database реализован по паттерну Singleton и предоставляет методы для добавления пользователей, проверки учётных данных, поиска по логину и email, а также обновления пароля. Для тестирования использовался фреймворк Qt Test. Набор тестов включал 9 тест-кейсов для класса Calculator, проверявших корректность вычисления каждой ветки кусочной функции, генерацию данных графика и обработку некорректных параметров. Также была

разработана таблица тест-кейсов из 13 сценариев, охватывающих регистрацию пользователей, авторизацию, сброс пароля, проверку блокировки при неверных OTP-кодах, выход из системы и корректность сброса состояния виджетов. Во время разработки проводилось тестирование основных функций системы: регистрация пользователей, авторизация с двухфакторной аутентификацией, отправка кодов через SMTP, сброс пароля, построение графика функции, защита от перебора OTP-кодов с блокировкой после 4 неверных попыток. Разработка тестовой документации позволила повысить стабильность и качество программного продукта.

В ходе выполнения проекта были изучены современные технологии разработки клиент-серверных приложений. Использование GitHub позволило организовать совместную разработку и контроль версий. Docker обеспечил контейнеризацию серверного компонента и упростил развертывание проекта. C++17 и Qt 5 использовались для разработки серверной и клиентской частей приложения. Doxygen автоматизировал создание технической документации и улучшил сопровождение кода. Работа с данными технологиями позволила получить практические навыки профессиональной разработки и лучше понять процессы создания современных программных систем.

Список использованных технологий: Git, GitHub, Docker, C++17, Qt 5 (Widgets, Network, SQL, Concurrent), SQLite, TCP/IP, SMTP (SSL), Doxygen, Qt Test, PlantUML.